



PAES-25: Local search, archiving, and multi/many-objective pseudo-boolean functions

Joshua Knowles, Arnaud Liefoghe

► To cite this version:

Joshua Knowles, Arnaud Liefoghe. PAES-25: Local search, archiving, and multi/many-objective pseudo-boolean functions. EMO 2025 – 13th International Conference on Evolutionary Multi-Criterion Optimization, Mar 2025, Canberra, Australia, France. pp.60-73, 10.1007/978-981-96-3506-1_5 . hal-04867338

HAL Id: hal-04867338

<https://hal.science/hal-04867338v1>

Submitted on 6 Jan 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons Attribution - NonCommercial 4.0 International License

PAES-25: Local Search, Archiving and Multi/Many-objective Pseudo-Boolean Functions

Joshua Knowles and Arnaud Liefooghe

¹ SLB, Cambridge, Cambridgeshire, UK

² LISIC, Université du Littoral Côte d’Opale, Calais, France

Abstract. Twenty-five years on from the original Pareto archived evolution strategy (PAES), we present and investigate an updated version (PAES-25), revisiting the algorithmic components of the mutation operator, acceptance criterion, and archiver, focusing on bit-string represented multi- and many- objective optimization problems. The original PAES, particularly the (1+1)-PAES, was intended as a “baseline” algorithm against which EMO algorithms (emerging at the time) with more parameters and the use of a population might be compared. PAES-25, which remains very simple, may serve similar purposes today, and may also help in developing our understanding of local search dynamics on multi-objective landscapes. Using LOTZ as a benchmark, and introducing three multi/many-objective variants, LITZ, sLITZ and FRITZ (up to 8 objectives here), the best performing PAES-25 configuration emerges as one using the multilevel grid archiver, a $1/n$ per-bit standard mutation, and original acceptance criterion (which accepts “neutral” search moves). We find no need for the use of hypervolume-based archiving, which is more computationally expensive, and generally recommend against an unbounded archive. Just as the original (1+1)-PAES has proven useful in developing purely Pareto-based hybrid EMO algorithms, as well as Pareto optimization local search algorithms like simulated annealing and tabu search variants, so should PAES-25, while now benefiting from an archiving component suitable for many-objective problems. We publish our functions, code, and results to facilitate future community benchmarking efforts.

Keywords: PAES, Pareto archived evolution strategy, local search, neighbourhoods, archiving, LOTZ function, benchmarking, many-objective methods.

1 Introduction

It is good to take stock periodically and reflect on the progress of the EMO field since its beginnings in 1985 (with VEGA [22]), through later periods of growth, proliferation of algorithms, and then integration, to where we are today. Many early ideas of the field are still found in modern algorithms, but hypothetically there has been development in theory and algorithm design that should result in improved performance — on benchmarks and hopefully in practice. Meanwhile, today’s benchmarking studies are more extensive and thorough than in

earlier times: they benefit from greater computational power, improvements to performance assessment, and more sophisticated test functions.

So how would an older evolutionary multi-objective optimization (EMO) algorithm fare when using some of the modern infrastructure (both algorithmic components and benchmarking methods) of today? We consider the (1+1)-PAES algorithm developed in 1999–2000 [6, 10]. Rather than conducting a large benchmarking study against many of today’s algorithms (which we leave for later work or invite others to pursue), we revisit the algorithm in its original form and also explore a small number of the most important potential improvements available today (without detracting from the essential core algorithm).

A key aspect of our study considers modern archiving theory and algorithms for multi-objective optimization. A recent 25-year survey [15] indicates that archiving theory started with PAES and related papers, and has developed gradually since then to have a broad impact. How can modern theory and algorithms inform and explain the performance of PAES when updating the original algorithm to incorporate alternative archivers available today? Additionally, we consider alternatives for the acceptance function in PAES, as well as the mutation (or neighbourhood) operator, drawing insights from recent research.

We call the resulting algorithm framework PAES-25 to mark the 25th anniversary of PAES. The original (1+1)-PAES algorithm was developed primarily as a response to the question, ‘what is the simplest, nontrivial equivalent of a random mutation hill climber for multi-objective optimization?’. The origin of this question is less familiar today, especially to younger researchers. At the time, it was something of a preoccupation of the (single-objective) EA field to try to “prove” the advantage of population-based methods (especially with crossover) over simple random mutation hill climbers [19] — and this turned out to be somewhat difficult to do. The authors of the original PAES wondered if the EMO algorithms of the time (MOGA, NPGA, original NSGA) would equally be on firm ground (or not) against a well thought-out simple hill climber for multi-objective problems (if one could be designed). The PAES paper (2000) investigated the (1+1)-PAES against similar algorithms with a population, i.e., $(\mu + \lambda)$ and (μ, λ) PAES variants, and the aforementioned EMO algorithms (also augmented with elitism and archiving). The result was that (1+1)-PAES fared rather well even though it had no mating population, did not ever select from the archive, and used only 1+1 random mutation-based search.

Although (1+1)-PAES performed comparatively well in the original study, it is to be expected that it might not work so well on many-objective optimization problems commonly considered today. Wishing to study the new PAES framework in relatively simple terms to begin with, we focus on discrete pseudo-Boolean functions with well-characterised (or derivable) properties that facilitate empirical analysis. For two-objective problems, we use the LOTZ function [13]. We also introduce new multi- or many-objective generalizations of the function (in Sect. 3). This allows us to study the behaviour of PAES in this more challenging setting and potentially also encouraging (EMO) running time theory, which has developed significantly in the period 2002–present.

Algorithm 1 PAES-25 (as original (1+1)-PAES but with plug-in archiver and neighbourhood)

Require: MO function $\mathbf{f}$, stopping criterion, neighbourhood
Require: archiver, acceptance rule, archive hyperparameter(s) ▷ see Sect. 2.1
 $A \leftarrow \emptyset$
Initialize a single current solution x
 $A \leftarrow A \cup \{(x, \mathbf{f}(x))\}$
while stopping criterion not met **do**
 $x' \leftarrow \text{neighbour}(x)$ ▷ see Sect. 2.2
 ArchiveUpdate($A, (x', \mathbf{f}(x'))$)
 if x' entered $A \wedge$ optional acceptance condition met **then** ▷ see Sect. 2.3
 $x \leftarrow x'$
 end if
end while
return(A)

Notwithstanding the original question that PAES was designed to answer (i.e., to propose a baseline against which EMO algorithms might be compared), it evidently emerged with several other uses. Hill climbers (or neighbourhood searchers) are components in many other meta-heuristics including tabu search, multi-start and iterated local search, simulated annealing, variable neighbourhood search, and memetic algorithms (a.k.a., hybrid EAs). The first author’s PhD thesis [9] proposed and investigated some of these variants of PAES (which uses Pareto comparisons only) for multi-objective optimization. The memetic variant M-PAES [11] was also later published in the literature.

Moreover, as (1+1)-PAES continued to perform rather well even in later studies (e.g., [3]), it serves not only as a baseline method for EMO algorithms to (try to) outperform in benchmarking studies, it also has some relevance as a method for use in practice. It has been included in several third-party software packages such as jMetal [21] or PlatEMO [23]. Arguably, such an algorithm with few parameters to tune and well-understood behaviour can be a useful choice for the “consumer”, to complement population-based methods with crossover that might require more tailoring to the target problem, or extensive tuning.

2 The PAES-25 Algorithm Framework

The PAES-25 framework proposed here in Algorithm 1 is identical to the original (1+1)-PAES provided that: (i) adaptive grid archiving (AGA) is used as the archiver, (ii) mutation is per-bit random mutation, and (iii) the acceptance rule always accepts a solution that has entered the archive. However, PAES-25 expands on this by including alternative choices for the archiver, mutation type, and acceptance rule (defined below).

2.1 Archivers

The primary purpose of archiving is to preserve nondominated solutions. (In PAES, this archive is also used to help make acceptance/rejection decisions).

An unbounded archiver (**UNB**) stores all such solutions. However, the archive can grow exponentially with the problem dimensions. To prevent this growth, a bounding mechanism can limit the archive size. Given an archive A and a maximum size μ , a bounded archiver returns a subset $A' \subseteq A$ such that $|A'| \leq \mu$; see [8, 15, 17]. PAES-25 can be used with UNB or with a bounded archiver, the following three being examples we recommend (and later study).

AGA: The adaptive grid archiver defined in [10] for the original (1+1)-PAES algorithm overlays an adaptive grid (or hypergrid) over the points in the current archive. When the archive is full and receives a new nondominated point, it admits the new point and removes one from the most crowded grid cell, breaking ties randomly. AGA was further analysed in [7], where a form of convergence was proved under mild conditions (for bi-objective problems).

HVA: Archiving aimed at maximizing the hypervolume of the PF approximation was first proposed in [7, 9] and was the EMO field’s first use of hypervolume for selection (here environmental selection) according to Bader [2]. When the archive is full and a new point is nondominated, HVA determines which of the $\mu + 1$ points contributes the least to the hypervolume. If the new point is the smallest contributor, it is discarded. Otherwise, it is admitted to the archive, replacing the least hypervolume contributor.

MGA: The multi-level grid archiver was first proposed in [14] and later analysed theoretically and empirically in [17]. Notably, MGA and HVA are the only known archivers that meet a number of desirable convergence properties (see *ibid.* for details and also [15]).

In particular, each subsequent archive is at least as good as previous ones in terms of Pareto dominance. MGA uses box-dominance, with box sizes determined by the smallest coarseness level that makes at least one point become box-dominated. A new non-dominated point is rejected if it falls within a dominated box. Otherwise, it is admitted and replaces a randomly chosen box-dominated point. HVA and MGA were both previously used in multi-objective local search in [18] for characterizing attraction points.

2.2 Mutation or Neighbourhood Operators

As alternative choices of mutation suitable for pseudo-Boolean functions, we consider the following three:

- k-flip:** Precisely k bits are flipped, chosen uniformly at random and with replacement (i.e., no memory of previous flips is used). We use $k = 1$ here.
- rate = 1/n:** Standard bit mutation with rate $1/n$; each bit is independently flipped with that probability, allowing for 1 or more bits to be flipped. (Our implementation uses rejection sampling in case no bits are flipped).

fast: A mixture distribution with heavy tails is used to first select the number k of bits to flip [4]. Then, the k -flip operator is applied. We follow the original setting, using a power-law distribution with $\beta = 1.5$.

2.3 Acceptance Rules

The standard acceptance rule of PAES-25 is to accept a solution if it has entered the archive, even if when equivalent solution (i.e., with same objective values) is already present. We call this approach “**neutral**” because it enables movement along neutral fitness plateaus. The alternative is to accept a solution only if it has both entered the archive and is unique within it. We label this “**strict**”.

3 Multi/Many-objective Pseudo-Boolean Test Functions

The use of test functions for the analysis of (proposed) EMO algorithms dates back to at least the mid-90s (see, e.g., [20]). Test function properties include the coding system used (binary, integer, or continuous variables), the number of objectives, and whether they are of “real-world” origin (e.g., derived from NP-hard problems [5] such as [16]) or of more theoretical interest. For theoretical problems, it is usually possible to establish the complete Pareto front (and Pareto set), allowing close analysis of algorithm performance. Further properties of interest include whether the function is scalable in the number of variables and/or objectives, correlations between objectives [18, 24], uni-modality vs. multi-modality, and other specific difficulties the function captures or emphasises (such as search bias).

LOTZ [13]

The LOTZ (Leading Ones Trailing Zeros) function introduced in [13] is a two-objective binary-variable unimodal problem with a scalable variable space. It has known Pareto front and set, and presents the difficulty that the Pareto front size scales linearly with the number of variables. LOTZ is also more complex than a simple counting-ones counting-zeros function where every string is Pareto optimal. It has been a favourite for advancing the theoretical analysis of EMO algorithms. We provide its definition below, closely following its original wording.

Definition 1. *The pseudo-Boolean function $\text{LOTZ} \{0, 1\}^n \rightarrow \mathbb{N}^2$ is computed as:*

$$\text{LOTZ}(\mathbf{x}) = \left(\sum_{i=1}^n \prod_{j=1}^i x_j, \sum_{i=1}^n \prod_{j=i}^n (1 - x_j) \right) .$$

A multi-objective version of the LOTZ function, mLOTZ [12], works by essentially copy-pasting the original LOTZ function multiple times along the string, i.e., by creating $m/2$ separate (separable) competing pairs of objectives. Interestingly, these pairs of objectives do not actually compete against each other. The function has also been of theoretical interest, even recently [25].

3.1 LITZ

The function we introduce, LITZ (standing for Leading Integers Trailing Zeros), is intended as an alternative multi-objective generalization of LOTZ.

Definition 2. *Assuming there are $m = 2^p, p > 2$ objectives, then the solution vector $\mathbf{x}$ is interpreted as an integer vector of length n/p , with each contiguous run of p bits defining the integer using standard binary encoding. The objectives are then calculated as follows:*

$$LI_k(\mathbf{x}) = \sum_{i=1}^{n/p} \prod_{j=1}^i I(\text{bin}_p((x_{j'}) = k)), \quad k = 1, 2, \dots, 2^p - 1, \quad j' = p \cdot (j - 1) + 1,$$

where $I(\cdot)$ is the indicator function, and $\text{bin}_p(x_{j'})$ returns the integer number corresponding to the binary value of the next p bits of $\mathbf{x}$ starting at index j' . The last objective is for trailing zeros:

$$TZ = \sum_{i=1}^{n/p} \prod_{j=i}^{n/p} I(\text{bin}_p((x_{j'}) = 0)), \quad j' = p \cdot (j - 1) + 1.$$

The observant reader may notice that LITZ captures some of the essence of LOTZ while having the property that at most one of the “leading integer” objectives can be non-zero for any bit string. We provide further properties of LITZ in Sect. 3.4.

3.2 sLITZ

To overcome the limitation on the number of objectives that can have non-zero values simultaneously (2 for LITZ), we introduce a variation below.

Definition 3. *The function sLITZ (standing for shuffled LITZ) is defined similarly to LITZ, except that the counts for the “leading” integers use a different permutation of the indexes of $\mathbf{x}$ for each objective. To keep the function regular, we choose permutations with starting points (from which to count leading integers) at $sp_k = 1 + n \cdot p \cdot ((k - 1) / (m - 1))$ on the binary coded string. For example, in a 4-objective sLITZ (which has 3 leading integers), the leading 1s are counted from position 1, but leading 3s are counted after position 2/4 of the way along the string. The remaining indexes shift accordingly modulo n , i.e., the permuted index i' of objective k is $1 + ((i + sp_k - 1) \bmod n)$.*

3.3 FRITZ

We introduce a last function, FRITZ, which stands for “First Run of Integers, Trailing Zeros.”

Definition 4. *FRI_k for $k \in \{1, \dots, m - 1\}$ is the length of the first run of consecutive integers k encountered, regardless of its starting position (counting from the bits encoding the first integer in increasing or left-to-right bit order). The TZ objective is computed as for LITZ.*

3.4 Analysis of the Pareto Fronts of LITZ, sLITZ and FRITZ

Theorem 1 (PF of LITZ). *The Pareto front of LITZ consists of the following set of points:*

$$\begin{aligned} &\{(0, 0, \dots, 0, n/p), (1, 0, \dots, 0, n/p - 1), \dots, (2, 0, \dots, 0, n/p - 2), \\ &\dots, (n/p, 0, \dots, 0, 0), (0, 1, \dots, 0, n/p - 1), \dots, \\ &(0, 2, \dots, 0, n/p - 2), \dots, (0, n/p, \dots, 0, 0), \dots\} \end{aligned}$$

Proof. It follows from the Pareto front of LOTZ and the definition of LITZ that each of the $m - 1$ objectives coding for non-zero integers takes the role of the 1s in LOTZ. Since only one of these objectives can be “leading”, all the other $m - 1$ of these has an evaluation of 0. $\square$

Theorem 2 (PF of sLITZ). *The Pareto front of sLITZ is a set of points that contains a number of subsets that are easily enumerable. The first such subset is $\{(f_1, \dots, f_m) \mid \sum_{i=1}^m f_i = n/p, \exists i, f_i = n/p\}$, representing each objective’s maximum score, with a score of zero for all other objectives. A second subset of the PF comprises all two-objective trade-offs between the first sLI objective and TZ: $\{(f_1, 0, \dots, 0, f_m) \mid f_1 + f_m = n/p\}$. Also a member of the PF is the “balanced point” $((n/p)/m, (n/p)/m, \dots, (n/p)/m)$. Other members of the PF depend upon more intricate conflicts between objectives; for example, $f_{m-1} = sLI_{m-1} + f_m = TZ \leq 2 \cdot (n/p)/m$ if $f_{m-1} > 0$ but certain other combinations for the other objectives can make up the sum of objectives to be n/p .*

Proof. We have constructed the PF algorithmically for the 96-bit problem and the 4- and 8-objective cases examined below, confirming the theorem. $\square$

Theorem 3 (PF of FRITZ). *The Pareto front of FRITZ is the set of points defined by $\{(k_1, \dots, k_m) \mid \sum_{i=1}^m k_i = n/p, \forall i, k_i \in 0..n/p\}$.*

Proof. It is clear that no run of positive integers or zeros can be greater than n/p , the length of the decoded binary string. Further, any of the positive integers or zero can be repeated throughout the length of the entire string. To obtain a member of the Pareto front starting from the all zeros string, simply substitute some of the zeros with runs of integers, starting from the left of the string, and never repeating a run of an integer already counted. Each substituted zero will be subtracting from the TZ objective while adding one to one of the FRI objectives. $\square$

We omit the characterization of the Pareto sets (the decision space pre-image of the PF) of each function due to space limitations.

4 Experimental Study & Results

We run several alternative variants of PAES-25 on a selection of the test functions presented above, as described in Table 1.

Table 1. Key parameter settings of our experiments. We ran all combinations of the settings shown, at different archive sizes (50, 100, 200 and 500), with 30 independent runs on each problem. Results are reported at different time steps, up to 2^{17} evaluations. In all cases, $n = 96$ bits.

Function		archivers	acceptance	mutation
LOTZ	$m = 2$	AGA, MGA, HVA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
LITZ	$m = 4$	MGA, HVA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
sLITZ	$m = 4$	MGA, HVA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
FRITZ	$m = 4$	MGA, HVA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
LITZ	$m = 8$	MGA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
sLITZ	$m = 8$	MGA, UNB	strict, neutral	1-flip, rate = $1/n$, fast
FRITZ	$m = 8$	MGA, UNB	strict, neutral	1-flip, rate = $1/n$, fast

Fig. 1 displays matrix plots of the results for the 2- and 4-objective problems, excluding FRITZ. The columns represent LOTZ, LITZ, and sLITZ, respectively. From top to bottom, the plots show: relative deviation of the current archive from the PF in terms of hypervolume (lower is better, with 0 indicating the entire PF has been found), archive size, number of encountered Pareto-optimal points, number of Pareto-optimal points in the current archive, number of accepted moves, last accepted move, and sum of objectives from the current solution. All statistics are plotted over time, providing a comprehensive view of algorithm performance.

A number of interesting observations emerge from these results. Most notably, the combination of standard $1/n$ mutation and neutral archiving consistently yields the best (or near-best) performance (assessed by hypervolume) across these problems. 1-flip mutation also performs reasonably well, but pairing it with *strict* acceptance appears to be a dangerous choice. This combination apparently gets stuck on local optima plateaus on many of the problems. Interestingly, fast mutation is not indicated on these functions. However, fast mutation (and $1/n$) are less affected by the choice between the neutral or strict acceptance rule, likely because they can inherently flip multiple bits at once and escape neutral plateaus that way.

Turning our attention to the archiver, we find that for these three problems (LOTZ with $m = 2$, and LITZ and sLITZ with $m = 4$), the choice of archiver is irrelevant. This is because the archive never reaches its chosen bound during the run. Consequently, we omit to show the performance of AGA and HVA for these problems, and to focus on a single archive size for ease of presentation. However, when looking at Fig. 2 for FRITZ (only) under various archivers and bounds, UNB yields superior performance. We attribute this to the large PF of FRITZ being accessible, especially for 1-flip mutation and neutral acceptance (here 1-flip is best). An unbounded archiver then allows for both a higher number of archived points and larger achieved hypervolumes.

For space reasons, we do not report CPU times for the PAES-25 variants. However, we observed that MGA runs significantly faster than HVA in most

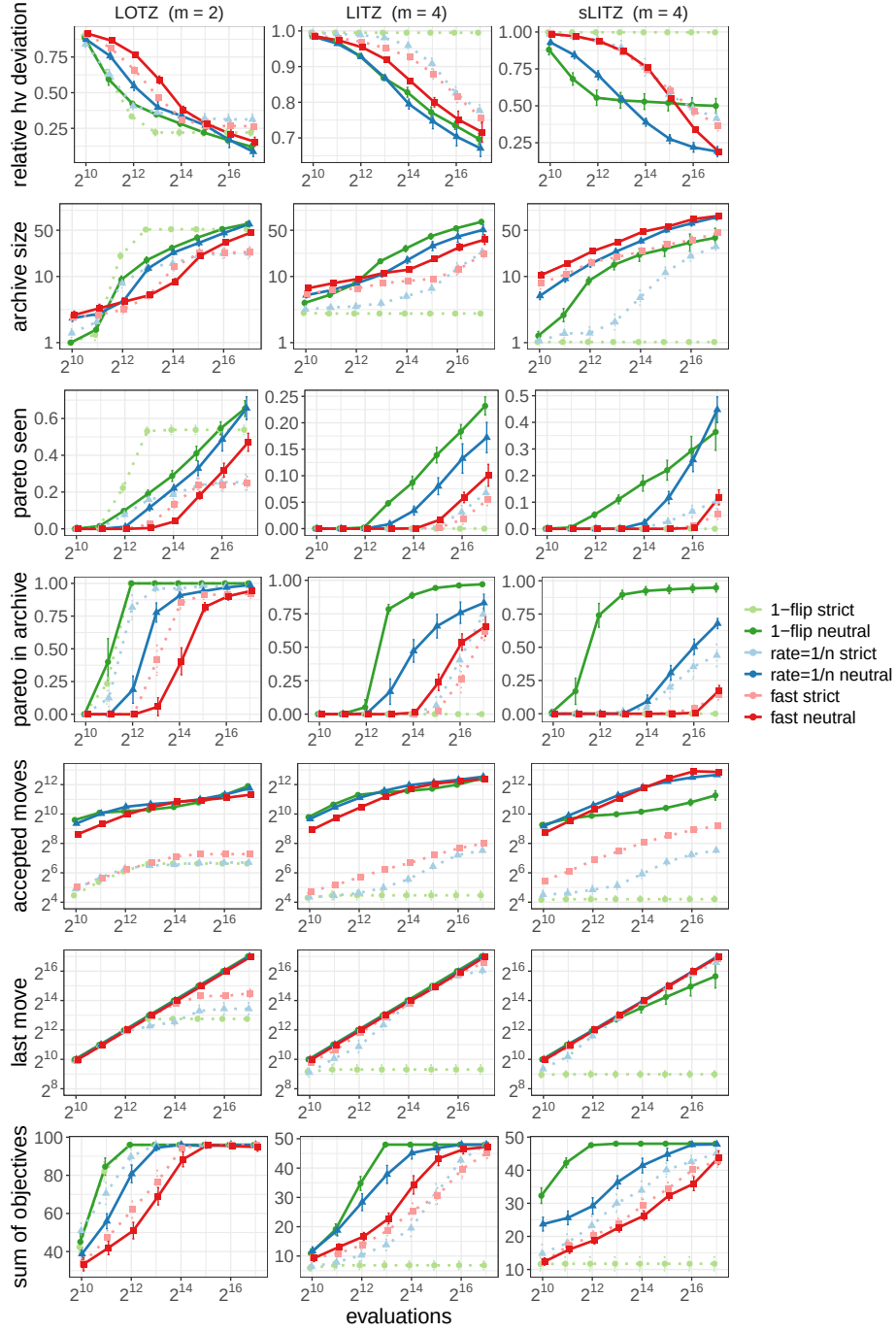


Fig. 1. Mean and 95% confidence intervals from 30 runs for LOTZ (2 objectives), LITZ, and sLITZ (both with 4 objectives) under MGA with a bound of 200.

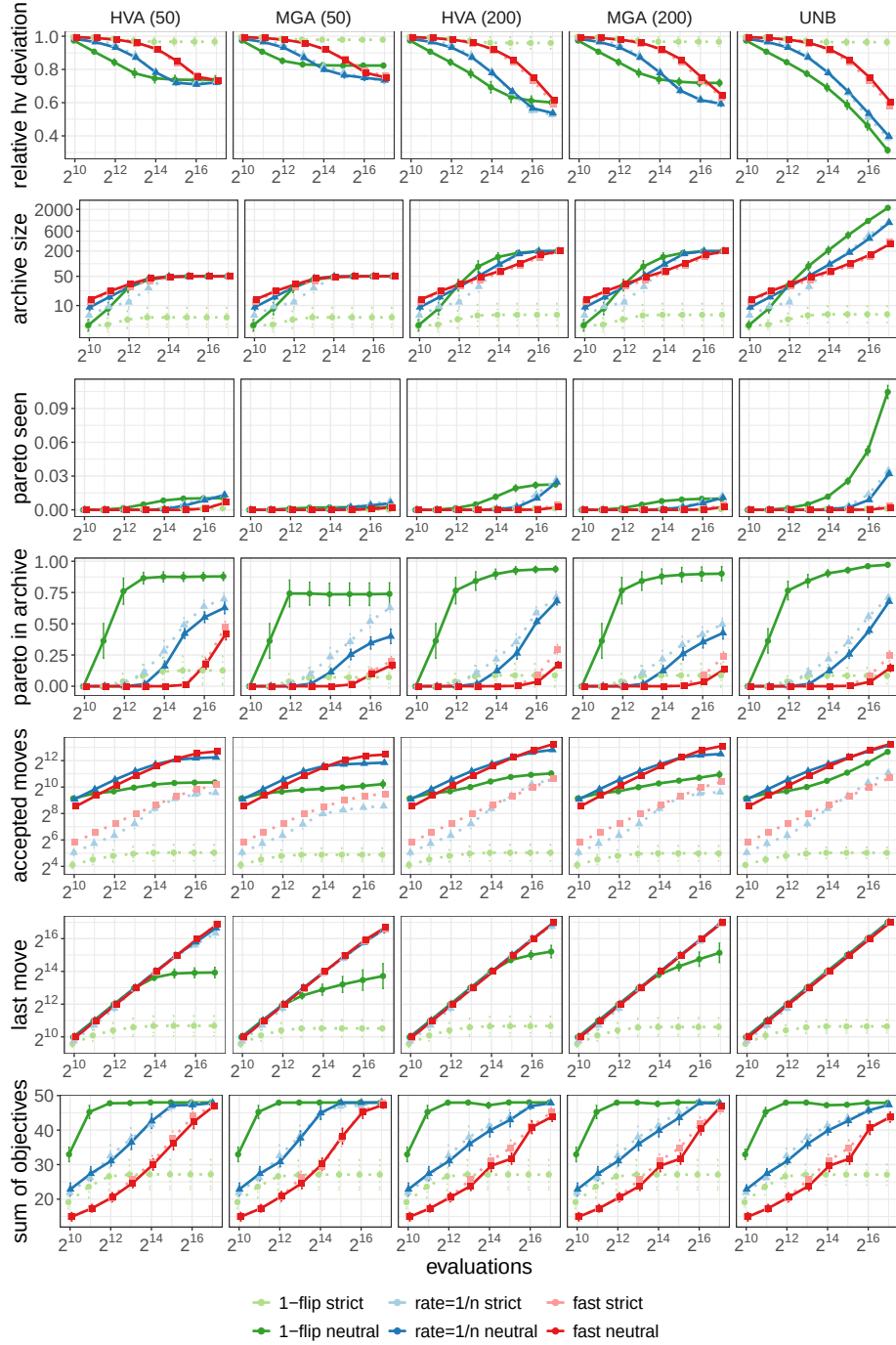


Fig. 2. Mean and 95% confidence intervals from 30 runs for FRITZ with 4 objectives under different archivers and bounds.

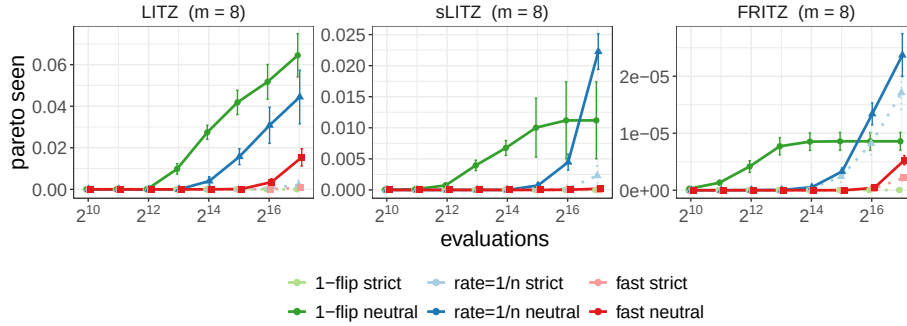


Fig. 3. Mean and 95% confidence intervals from 30 runs for LITZ, sLITZ, and FRITZ with 8 objectives under MGA with a bound of 200.

problem configurations, even with small archives. The FRITZ problem, in particular, causes substantial overhead for HVA. For the 8-objective problems, we could not complete 30 independent runs after several days, while all other algorithm runs finished within minutes or hours. The UNB archiver is computationally more expensive to update than MGA for FRITZ. This is expected and due to the large number of archived points, which requires numerous Pareto comparisons each time a point is considered for archive entry.

A small sample of the results on the 8-objective problems is given in Fig. 3 under a specific archiver setting. These findings align with previous results, particularly in how 1-flip mutation tends to find more Pareto-optimal solutions than standard mutation, except for sLITZ and FRITZ under larger budgets. Complete results, including visualizations of solutions in the objective space, are available in the supplementary material: <https://doi.org/10.5281/zenodo.13889583>.

5 Concluding Discussion

The (1+1)-PAES algorithm [10] was arguably the first multi-objective optimization algorithm designed to work based on local search and Pareto comparisons only [16]. Diversity in the Pareto front approximation is also pursued in PAES through its original archiver (AGA). Its modular design, separating a basic mutation-based hillclimber from comparisons against an archive representing the Pareto front approximation accumulated so far, has since been emulated by many other multi-objective evolutionary and local search algorithms.

Subsequent to the original PAES algorithm, developments in multi-objective evolutionary and local search algorithms have revisited and potentially improved upon some of its components. Meanwhile, there has been increasing interest in so-called many-objective optimization — problems with more than 3 objectives. These developments warrant careful evaluation of progress, which may be facilitated by advances in theory and/or empirical benchmarking methods.

To this end, we have proposed PAES-25, a modular version of PAES that explicitly allows for different choices of archiver, mutation, and acceptance func-

tions. Given this slightly broader framework, how should we select these components? Are these choices straightforward and understandable — at least when properties of the function to be optimized are known?

Our experiments examined simple pseudo-Boolean functions, and even here, the interplay between archiver, mutation and acceptance gave subtly different dynamics depending on the problem. Nevertheless, an overall finding emerged: the combination of a standard $1/n$ mutation and neutral acceptance rule generally provided a stable setting here. A so-called “fast mutation” did not perform well on any of the problems studied here, and this follows expectation. Perhaps the marked drop-off in performance is larger than anticipated, however.

The best choice of archiver is slightly more subtle. AGA was not designed for many objectives, and its convergence proof was only given for the 2-objective case. Arguably, it has been surpassed by later developments, and so we focused on the three other archivers, MGA, HVA and UNB. Comparing MGA and HVA, we find that MGA performs equally well in interaction with the search process. However, it is significantly less computationally expensive, even when using modern hypervolume computation algorithms.

Between MGA and UNB, the choice seems more a philosophical one — or a personal preference based on context. If you find it acceptable to present potentially billions of solutions to a user, or if you believe that post-processing of a large archive can be done at the end of a run before presenting to a decision maker, then the unbounded archive does sometimes produce better PF approximations than MGA. However, this comes at the cost of increasingly expensive Pareto comparisons as the archive size grows. We did not study here the effect of fast comparison methods that have been proposed elsewhere [1], so this observation needs qualification and further investigation. However, we note that on a function like FRITZ, we did not finish approximating the PF even in 2^{17} function evaluations. Given these considerations, we can tentatively say that MGA seems an excellent compromise and very much aligns with the original PAES algorithm’s use of a fast, cell-based archiver to approximate the PF discovered.

Given the popularity and longevity of the original (1+1)-PAES, we believe that this new updated version, PAES-25, may be an important algorithm for future study. We have generalized it, with MGA, to an algorithm that we have demonstrated works on many-objective problems. Our initial findings also show neutral acceptance and standard $1/n$ mutation are good choices generally. This results in an algorithm with very few parameters and low coding complexity. We hope that PAES-25 will find a place in future theoretical and benchmarking studies as a baseline. We also expect it to be a useful and valuable addition to some of the popular EMO packages (mentioned previously). Facilitating further study, we have made results and benchmarks available at <https://doi.org/10.5281/zenodo.13889583>.

Acknowledgments. JK wishes to thank SLB for supporting the publication of this work. JK also acknowledges the contributions of David Corne to the original PAES and related work, and thanks him for useful discussions. Both authors gratefully acknowledge the Lorentz centre workshop on Benchmarking in Multi-Criteria Optimization (BeMCO 2024), which sparked initial discussions for this paper.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Allmendinger, R., Jaskiewicz, A., Liefoghe, A., Tammer, C.: What if we increase the number of objectives? Theoretical and empirical implications for many-objective combinatorial optimization. *Computers & Operations Research* **145**, 105857 (2022), <https://doi.org/10.1016/j.cor.2022.105857>
- [2] Bader, J.M.: Hypervolume-based search for multiobjective optimization: theory and methods. Ph.D. thesis, ETH Zurich (2010)
- [3] Coello Coello, C.A.: Evolutionary multi-objective optimization: a historical view of the field. *IEEE Computational Intelligence Magazine* **1**(1), 28–36 (2006)
- [4] Doerr, B., Le, H.P., Makhmara, R., Nguyen, T.D.: Fast genetic algorithms. In: *Proceedings of the genetic and evolutionary computation conference*, pp. 777–784 (2017)
- [5] Garey, M., Johnson, D.: *Computers and intractability: a guide to the theory of NP-Completeness*. WH Freeman and Co. (1979)
- [6] Knowles, J., Corne, D.: The Pareto archived evolution strategy: A new baseline algorithm for pareto multiobjective optimisation. In: *Proceedings of the 1999 congress on evolutionary computation-CEC99* (Cat. No. 99TH8406), vol. 1, pp. 98–105, IEEE (1999)
- [7] Knowles, J., Corne, D.: Properties of an adaptive archiving algorithm for storing nondominated vectors. *IEEE Transactions on Evolutionary Computation* **7**(2), 100–116 (2003)
- [8] Knowles, J., Corne, D.: Bounded pareto archiving: Theory and practice. In: *Metaheuristics for multiobjective optimisation*, pp. 39–64, Springer Berlin Heidelberg (2004)
- [9] Knowles, J.D.: *Local Search and Hybrid Evolutionary Algorithms for Pareto Optimization*. Ph.D. thesis, Department of Computer Science, University of Reading, UK (2002)
- [10] Knowles, J.D., Corne, D.: Approximating the nondominated front using the Pareto archived evolution strategy. *Evolutionary Computation* **8**(2), 149–172 (2000), <https://doi.org/10.1162/106365600568167>
- [11] Knowles, J.D., Corne, D.: M-PAES: A memetic algorithm for multiobjective optimization. In: *Proceedings of the 2000 Congress on Evolutionary Computation (CEC'00)*, pp. 325–332, IEEE Press, Piscataway, NJ (Jul 2000)
- [12] Laumanns, M., Thiele, L., Zitzler, E.: Running time analysis of multiobjective evolutionary algorithms on pseudo-boolean functions. *IEEE Transactions on Evolutionary Computation* **8**(2), 170–182 (2004)
- [13] Laumanns, M., Thiele, L., Zitzler, E., Welzl, E., Deb, K.: Running time analysis of multi-objective evolutionary algorithms on a simple discrete optimization problem. In: *Parallel Problem Solving from Nature—PPSN VII: 7th International Conference Granada, Spain, September 7–11, 2002 Proceedings 7*, pp. 44–53, Springer (2002)

- [14] Laumanns, M., Zenklusen, R.: Stochastic convergence of random search methods to fixed size pareto front approximations. *European Journal of Operational Research* **213**(2), 414–421 (2011)
- [15] Li, M., López-Ibáñez, M., Yao, X.: Multi-objective archiving. *IEEE Transactions on Evolutionary Computation* **28**, 696–717 (2024)
- [16] Liefvooghe, A., Humeau, J., Mesmoudi, S., Jourdan, L., Talbi, E.G.: On dominance-based multiobjective local search: design, implementation and experimental analysis on scheduling and traveling salesman problems. *Journal of Heuristics* **18**(2), 317–352 (2012), <https://doi.org/10.1007/s10732-011-9181-3>
- [17] López-Ibáñez, M., Knowles, J., Laumanns, M.: On sequential online archiving of objective vectors. In: *Evolutionary Multi-Criterion Optimization (EMO 2011)*, Lecture Notes in Computer Science, vol. 8672, pp. 46–60, Springer (2011)
- [18] López-Ibáñez, M., Liefvooghe, A., Verel, S.: Local optimal sets and bounded archiving on multi-objective NK-landscapes with correlated objectives. In: Bartz-Beielstein, T., Branke, J., Filipič, B., Smith, J. (eds.) *Parallel Problem Solving from Nature – PPSN XIII*, Lecture Notes in Computer Science, vol. 8672, pp. 621–630, Springer, Heidelberg, Germany (2014), https://doi.org/10.1007/978-3-319-10762-2_61
- [19] Mitchell, M., Holland, J., Forrest, S.: When will a genetic algorithm outperform hill climbing? *Advances in Neural Information Processing Systems* **6** (1993)
- [20] Murata, T., Ishibuchi, H., et al.: MOGA: multi-objective genetic algorithms. In: *IEEE international conference on evolutionary computation*, vol. 1, pp. 289–294, IEEE Piscataway (1995)
- [21] Nebro, A.J., Durillo, J.J., Vergne, M.: Redesigning the jMetal multi-objective optimization framework. In: Jiménez Laredo, J.L., Silva, S., Esparcia-Alcázar, A.I. (eds.) *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO Companion 2015*, pp. 1093–1100, ACM Press, New York, NY (2015), <https://doi.org/10.1145/2739482.2768462>
- [22] Schaffer, J.D.: Some experiments in machine learning using vector evaluated genetic algorithms. Tech. rep., Vanderbilt Univ., Nashville, TN (USA) (1985)
- [23] Tian, Y., Cheng, R., Zhang, X., Jin, Y.: PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum]. *IEEE Computational Intelligence Magazine* **12**(4), 73–87 (2017), <https://doi.org/10.1109/MCI.2017.2742868>
- [24] Verel, S., Liefvooghe, A., Jourdan, L., Dhaenens, C.: On the structure of multiobjective combinatorial search space: *MNK*-landscapes with correlated objectives. *European Journal of Operational Research* **227**(2), 331–342 (2013)
- [25] Zheng, W., Doerr, B.: Runtime analysis for the NSGA-II: proving, quantifying, and explaining the inefficiency for many objectives. *IEEE Transactions on Evolutionary Computation* pp. 1–1 (2023), <https://doi.org/10.1109/TEVC.2023.3320278>